



Bangladesh University of Engineering and Technology
Department of Electrical and Electronic Engineering
Dhaka-1205, Bangladesh

Digital Signal Processing I Laboratory

EEE 312

PROJECT REPORT

Security System for Bank Based on Voice Recognition using MATLAB

Drive link: [Project Demonstration](#)

Prepared By:

Mushfiqur Rahman ID: 2106181	Chayan Paul ID: 2106182	Kayes Sami Malitha ID: 2106184
---------------------------------	----------------------------	-----------------------------------

Submitted to:

Shafin Bin Hamid

Assistant Professor, Department of EEE, BUET

Rafid Hassan Palash

Lecturer, Department of EEE, BUET

Abstract:

The main goal of this project is to design a Secure and Automated bank access system using voice recognition technology. The system uses MATLAB to process and analyze voice recordings. By extracting voice features like Mel-Frequency Cepstral Coefficients (MFCCs)^[1] and analyzing them using machine learning models (such as Random Forest)^[2], the system can identify whether a person is the real account holder or not.

Each user's spoken name and ID are recorded, processed, and stored in a voice database. When someone tries to access the system, their current voice input is compared to these stored patterns. If the match is successful, access is granted.

Multiple experiments show that this system performs very accurately, especially in quiet environments. It outperforms traditional security methods like PIN codes or fingerprints because it uses unique voice characteristics that are difficult to copy or fake.

Introduction:

In today's banking landscape, fraud and identity theft are increasing rapidly, making traditional security measures like account numbers or PINs less reliable. Since an account number can be easily shared or guessed, relying solely on such information no longer ensures protection of a user's assets.

To address this issue, our project introduces a voice-based authentication system. Instead of relying on text passwords or cards, this system listens to the spoken name and ID of the user, then determines whether that voice truly belongs to the account holder. The idea is simple but powerful because voice becomes key.

The system is both speaker-dependent and speech-dependent, meaning it checks not just what is being said (name and ID) but also who is saying it. When a user speaks into the system, the voice input is recorded and analyzed in MATLAB. Key features are extracted using the MFCC (Mel-Frequency Cepstral Coefficients) technique, which captures the unique vocal fingerprint of the person. These features are then matched against a stored database of legitimate users.

To perform this comparison, we trained machine learning models (RF classifiers) that learned to distinguish between different users based on their voice patterns. Both name and ID are verified separately, and access is granted only if both predictions match the person's credentials.

Since every person's voice is unique, it can be used as a secure way to confirm identity. Even twins don't have identical voices; this system offers a much higher level of security than conventional methods. Also, since no physical device like a card or fingerprint scanner is required, the process is faster, more hygienic, and user-friendly.

In conclusion, this project not only strengthens bank security but also localizes technology, making biometric authentication more accessible and effective.

System Architecture:

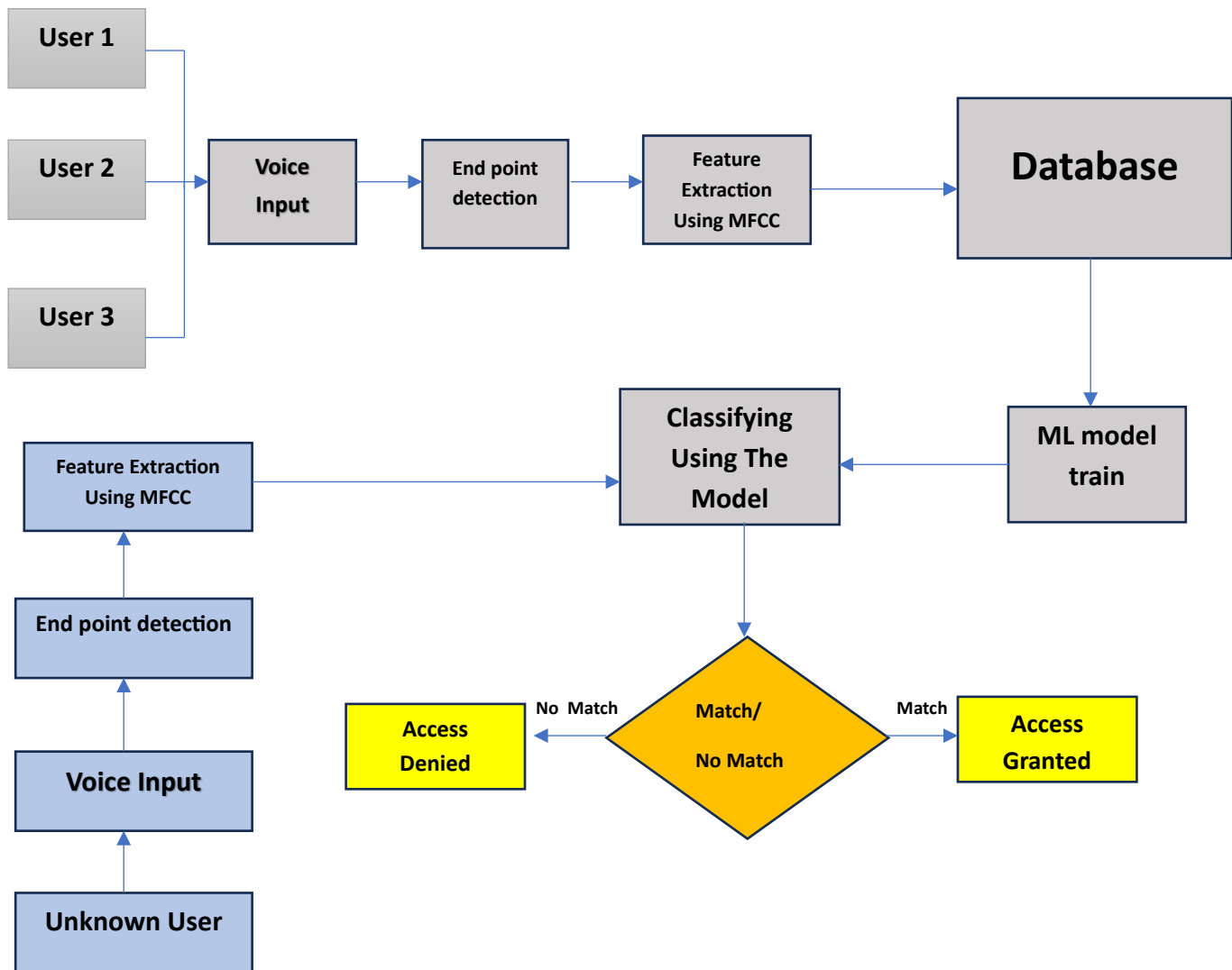


Fig. 1: System Architecture

Methodology:

a. Data Collection - Matlab file `_data_collection.m`

Explanation: This MATLAB code serves as a data collection module for a voice recognition project using an SVM model. It interactively records audio samples from users, where each user provides their ID and the number of samples to record. For every user, the system captures two types of voice inputs: one where the user states their **Name** and another where they state their **ID (last 3 digits)**. These recordings are made using MATLAB's audiorecorder and saved as **.wav** files with a structured filename (**<userID>_<sample number>.wav**). Name samples are stored in the **NAME_DATABASE** folder, while ID samples go to the **ID_DATABASE** folder. The recording process is repeated for the given number of samples and can continue for multiple users based on user input. This organized dataset of voice recordings will later be used for extracting features like MFCCs and training the SVM classifier to recognize speakers based on their voice patterns.

For our entire project, we used a 44.1kHz sampling rate. For encoding, 8 bits per sample is used. And the voice recording duration is set to 4 seconds.

b. End Point Detection – Matlab file `_endpointdetection.m`

Explanation: This function performs **endpoint detection**, which is crucial in your voice recognition project to isolate the actual spoken part from silence or background noise. It first applies **Wiener filtering**^[3] to reduce noise. Then, it analyzes the signal using two features: **Short-Time Energy (STE)**^[4], which measures the signal's loudness or intensity over short frames and is higher during speech; and **Zero-Crossing Rate (ZCR)**^[4], which counts how often the signal changes sign and tends to be higher during voiced speech due to rapid frequency changes. The function slides through the signal in overlapping frames, computes STE and ZCR for each, and then uses thresholds based on their peak values to detect frames that contain real speech. It returns only the part of the signal between the first and last detected voiced frames. This ensures that only the relevant speech portion is passed to later stages like MFCC extraction and ML classification, improving both accuracy and efficiency of the system.

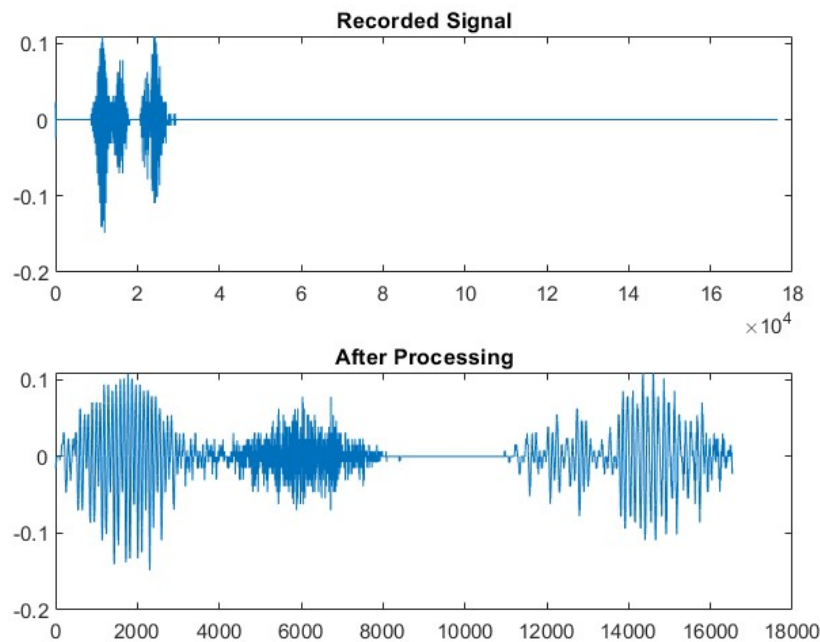


Fig. 2: Output After Endpoint Detection

c. Database Generation – Matlab file - database_generator.m

Explanation: This code loads the voice samples from the corresponding folder and extracts the name of the audio file, which is used to label the corresponding feature column. **MFCC (Mel-frequency cepstral coefficients)** coefficients are calculated using MATLAB's built-in function `mfcc()`. We considered 13 coefficients along with the 0th coefficient for our project. Code segment -

```
speechSegment = endpointdetection(s, fs);
% Normalize amplitude to (-1,1) range
end1 = speechSegment / max(abs(speechSegment));
% Extract MFCCs
coeffs = mfcc(end1, fs, 'NumCoeffs', 13);

% Normalize MFCCs
coeffs_norm = (coeffs - mean(coeffs,1)) ./ std(coeffs,[],1);

% Calculate delta and delta-delta
delta = diff([coeffs_norm(1,:); coeffs_norm],1,1); % first derivative
delta = [delta; delta(end,:)]; % ensuring same size as coeffs
deltaDelta = diff([delta(1,:); delta],1,1); % second derivative
deltaDelta = [deltaDelta; deltaDelta(end,:)]; % ensuring same size as coeffs
```

```
% Combine features by averaging over frames
```

```
featureVector = [mean(coeffs_norm,1), mean(delta,1), mean(deltaDelta,1)];
```

Delta (first derivative) and **delta-delta** (second derivative) features are calculated using the normalized MFCC coefficients. Normalization ensures features are on a comparable scale (mean = 0, std = 1), which helps most machine learning algorithms perform better. So, we took 14 MFCC (zeroth coefficient included), 14 delta & 14 delta-delta coefficients. In total, 42 coefficients are extracted and saved in the database for each voice sample.

The database is then saved to the file `DATABASE.m` with name data and id data.

MAT-file containing:			id_data name_data				
Name	Size	Bytes Class	Columns 1 through 5				
id_data	43x15	82650 cell array					
name_data	43x15	82650 cell array					
			{'2106181' }	{'2106181' }	{'2106181' }	{'2106181' }	{'2106181' }
			{[-5.3597e-17]}	{[-7.9302e-18]}	{[1.4663e-17]}	{[-7.1371e-17]}	{[9.4452e-17]}
			{[-3.0627e-17]}	{[-1.0137e-15]}	{[-4.6294e-16]}	{[6.6296e-16]}	{[-5.6340e-17]}
			{[-3.0627e-17]}	{[1.3845e-16]}	{[1.3249e-16]}	{[5.9476e-17]}	{[-3.3141e-17]}
			{[-4.5940e-17]}	{[2.8549e-16]}	{[6.8080e-17]}	{[8.2474e-17]}	{[1.9885e-17]}
			{[2.6798e-17]}	{[-2.1147e-16]}	{[-1.4244e-16]}	{[1.6495e-16]}	{[6.2968e-17]}
			{[-1.5313e-17]}	{[-3.8329e-17]}	{[-3.6658e-18]}	{[3.1721e-18]}	{[-7.9538e-17]}
			{[-7.6567e-18]}	{[-9.5162e-17]}	{[-1.1312e-16]}	{[6.9785e-16]}	{[3.4798e-17]}
			{[2.2970e-17]}	{[4.3616e-16]}	{[-3.3516e-17]}	{[-4.1475e-16]}	{[-3.3141e-18]}
			{[-1.1485e-17]}	{[2.4584e-16]}	{[1.2150e-16]}	{[-1.5226e-16]}	{[7.6224e-17]}
			{[7.6567e-18]}	{[7.9302e-18]}	{[-2.5137e-17]}	{[-8.6042e-17]}	{[3.9769e-17]}
			{[7.6567e-17]}	{[1.5860e-17]}	{[-5.0274e-17]}	{[-6.3441e-18]}	{[-3.9769e-17]}
			{[1.7993e-16]}	{[-2.0486e-17]}	{[-1.6758e-17]}	{[-2.2204e-17]}	{[-6.4625e-17]}
			{[-1.3399e-17]}	{[-3.1721e-17]}	{[-1.0474e-16]}	{[6.9785e-17]}	{[8.6167e-17]}
			{[5.7425e-18]}	{[2.6434e-17]}	{[-1.6758e-16]}	{[-1.0151e-16]}	{[-3.4798e-17]}
			{[-1.5778e-04]}	{[-7.2694e-04]}	{[-0.0042]}	{[0.0033]}	{[-1.7648e-05]}
			{[-4.4769e-05]}	{[0.0039]}	{[-0.0033]}	{[0.0134]}	{[-2.2740e-05]}
			{[-0.0101]}	{[-0.0273]}	{[-0.0249]}	{[-0.0333]}	{[-0.0243]}
			{[-0.0484]}	{[-0.0180]}	{[-0.0103]}	{[-0.0413]}	{[0.0191]}
			{[0.0438]}	{[0.0331]}	{[0.0282]}	{[0.0361]}	{[0.0268]}
			{[0.0407]}	{[0.0217]}	{[0.0345]}	{[0.0327]}	{[0.0025]}
			{[-0.0435]}	{[-0.0120]}	{[0.0311]}	{[-0.0138]}	{[-0.0122]}
			{[-0.0049]}	{[0.0037]}	{[0.0230]}	{[0.0035]}	{[-0.0027]}

Fig. 3: Database Preview

d. Model Train – Matlab file - `model_train.m`

Explanation: Here, we first load the saved dataset and extract the features and labels for the corresponding sample and save them in separate vectors for model training. Labels are then converted to categorical values. These features and labels data are then fed into the machine learning models. We trained KNN (k nearest neighbour), SVM (support vector machine), and RF (random forest) models separately. Code segment-

```

%% Random Forest models
% Define template for bagged trees
t = templateTree('MaxNumSplits', 20, 'MinLeafSize', 5); % Initial settings
hyperopts = struct(...
    'AcquisitionFunctionName', 'expected-improvement-plus', ...
    'MaxObjectiveEvaluations', 30, ... % Limit iterations for tuning
    'ShowPlots', false, ... % Display optimization progress
    'Verbose', 1,... % Show progress in command window
    'KFold', 5);

% Define parameters to optimize
paramToOptimize = {...
    'NumLearningCycles', 'MaxNumSplits', 'MinLeafSize'};

% Train ensemble with hyperparameter optimization
ml_name = fitcensemble(features_name_train, string(labels_names_train), ...
    'Method', 'Bag', ...
    'Learners', t, ...
    'OptimizeHyperparameters', paramToOptimize, ...
    'HyperparameterOptimizationOptions', hyperopts);

ml_id = fitcensemble(features_id_train, string(labels_ids_train), ...
    'Method', 'Bag', ...
    'Learners', t, ...
    'OptimizeHyperparameters', paramToOptimize, ...
    'HyperparameterOptimizationOptions', hyperopts);

```

We trained two models, one for name recognition and another for id recognition. We used hyperparameter tuning for both of them and optimized 'NumLearningCycles', 'MaxNumSplits', and 'MinLeafSize'. 5-fold cross-validation was used to ensure robust optimization.

These models are then saved to `ml_model.m` MATLAB file.

e. Identification – MATLAB file - `voice_identification.m`

Explanation: This code takes voice input from an unknown user, extracts the MFCC features, uses these features to feed into the train model and predicts the user. If the prediction is matched with the user id input, then access is granted; otherwise denied.

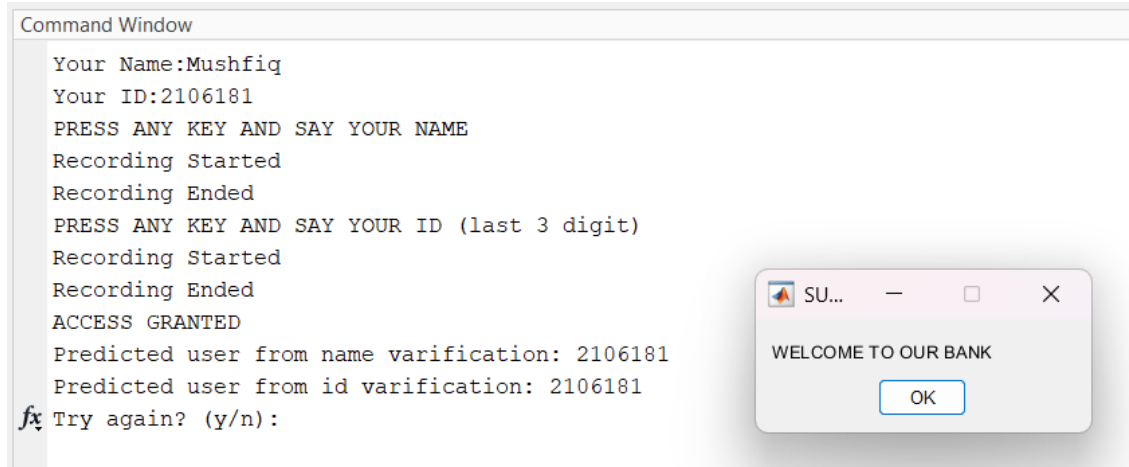


Fig.4: Identification Window

f. Model Evaluation – MATLAB file - model_evaluation.m & confusionmatStats.m

Explanation: We separately took voice samples and generated a dataset to perform test analysis on the trained models. The file **model_evaluation.m** is used to generate a confusion matrix and a classification report for the test data. We have taken the function **confusionmatStats.m**^[5] from the MATLAB file exchange to compute the classification report.

Performance Analysis:

KNN Model:

We got cross-validation accuracy of 100% for name model and 86.67% for id model.

Classification report-

Command Window

```
--- Classification Report For Name Model---
  Class      Precision      Recall      F1score      Accuracy
  -----
  2106181      0.83333          1      0.90909      0.93333
  2106182          1          1          1          1
  2106184          1          0.8      0.88889      0.93333

Macro Average Precision: 0.94
Macro Average Recall:    0.93
Macro Average F1-score:  0.93

--- Classification Report For Id Model---
  Class      Precision      Recall      F1score      Accuracy
  -----
  2106181      0.83333          1      0.90909      0.93333
  2106182      0.66667          0.8      0.72727          0.8
  2106184          1          0.6          0.75      0.86667

Macro Average Precision: 0.83
Macro Average Recall:    0.80
Macro Average F1-score:  0.80

fx >>
```

Fig.5: Classification Report of KNN Model

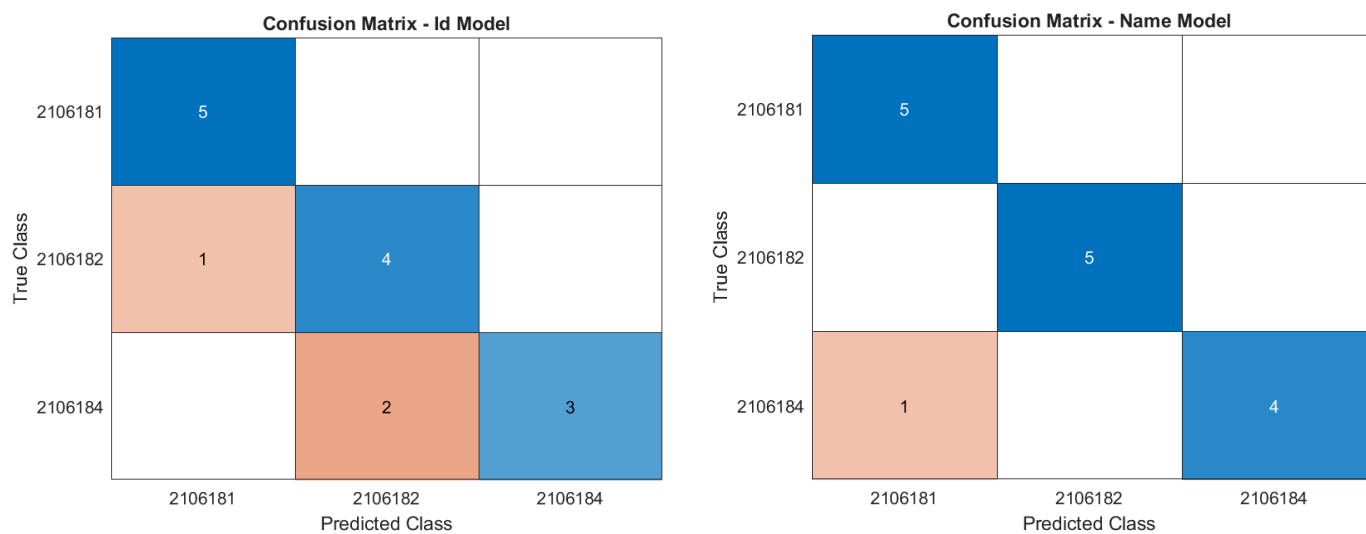


Fig.6: Confusion Matrix of KNN Model

SVM Model:

The cross-validation score for this model was 80% (name model) & 93.33% (id model).

Classification report-

Command Window

```
--- Classification Report For Name Model---
  Class      Precision      Recall      F1score      Accuracy
  -----
  2106181      0.83333          1          0.90909      0.93333
  2106182          0.8          0.8          0.8      0.86667
  2106184          0.75          0.6          0.66667          0.8

Macro Average Precision: 0.79
Macro Average Recall:    0.80
Macro Average F1-score: 0.79

--- Classification Report For Id Model---
  Class      Precision      Recall      F1score      Accuracy
  -----
  2106181      0.66667          0.8          0.72727          0.8
  2106182          1          0.2          0.33333      0.73333
  2106184          0.625          1          0.76923          0.8

Macro Average Precision: 0.76
Macro Average Recall:    0.67
Macro Average F1-score: 0.61
```

```
fx >> |
```

Fig.7: Classification Report of SVM Model

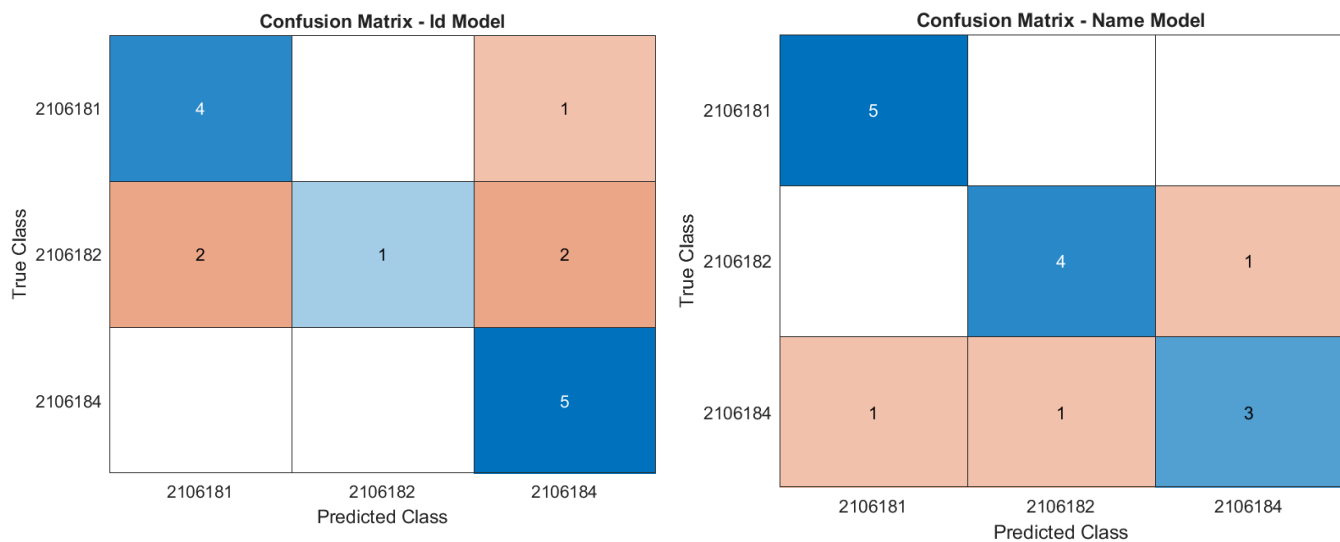


Fig.8: Confusion Matrix of SVM Model

RF Model:

We got cross-validation accuracy of 93.33% for name model and 86.67% for id model.

Classification report-

```
Command Window
```

```
--- Classification Report For Name Model---
```

Class	Precision	Recall	F1score	Accuracy
2106181	1	1	1	1
2106182	1	1	1	1
2106184	1	1	1	1

```
Macro Average Precision: 1.00  
Macro Average Recall: 1.00  
Macro Average F1-score: 1.00
```

```
--- Classification Report For Id Model---
```

Class	Precision	Recall	F1score	Accuracy
2106181	1	0.8	0.88889	0.93333
2106182	0.83333	1	0.90909	0.93333
2106184	1	1	1	1

```
Macro Average Precision: 0.94  
Macro Average Recall: 0.93  
Macro Average F1-score: 0.93
```

```
fx >>
```

Fig.9: Classification Report of RF Model

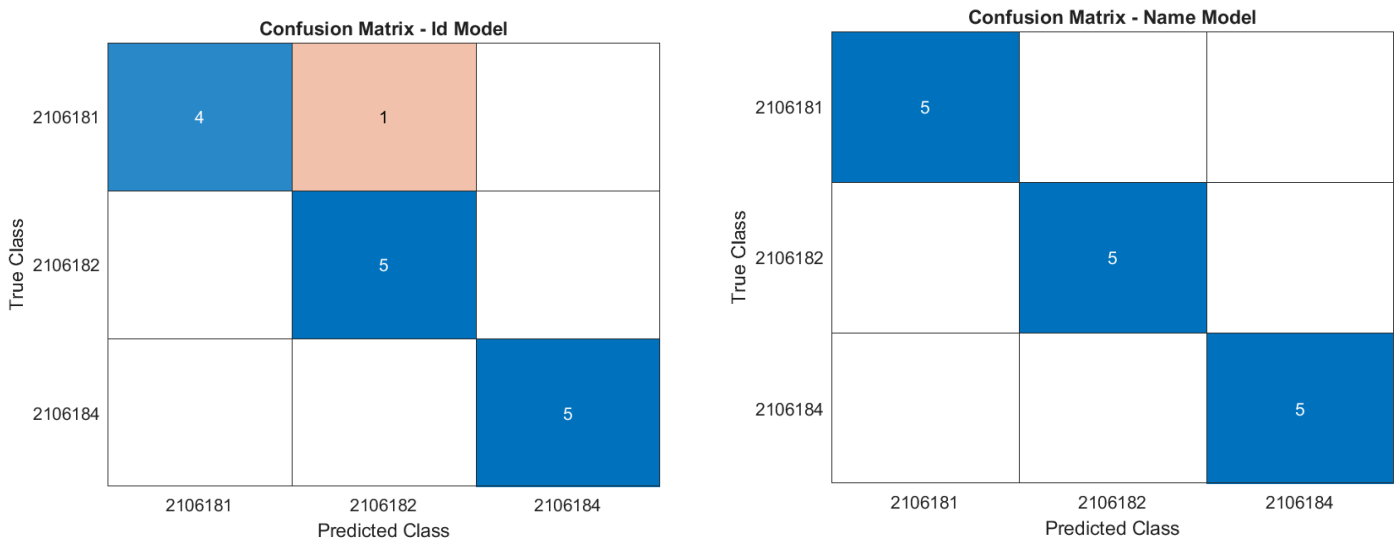


Fig.10: Confusion Matrix of RF Model

Model Selection:

As can be seen from the evaluation parameters, the RF (random forest) model performed best. We also tested in real time and got the best result from RF model.

For the other two models, we obtained a high cross-validation score for the KNN model, but in real-time usage, it performed poorly. Conversely, the SVM model had a low cross-validation score but performed satisfactorily in real-time applications.

Considering all the parameters, we selected the RF model as it performed the best.

Advantages:

Advantages of our proposed method of voice identification are-

- **Biometric Security:**
Provides secure, speaker-specific identification without the need for physical biometrics like fingerprints or retina scans.
- **Language-Localized:**
Supports Bengali voice inputs, making it user-friendly and accessible for native speakers in local or regional applications.
- **Contactless and Convenient:**
Eliminates the need for carrying physical ID cards, tokens, or typing credentials—authentication is done simply by speaking.
- **Scalability and Flexibility:**
The system can be extended to other domains like ATM authentication, secure login systems, and voice-controlled banking, offering a versatile and modern approach to user verification.

Limitations:

There are some limitations of this project-

- **Noise Sensitivity:**
The system may perform poorly in noisy environments, where background sounds can interfere with accurate voice recognition.
- **Accent and Pronunciation Variations:**
Variations in accents, speech speed, or pronunciation among different users may reduce recognition accuracy.

- **Data Collection Limitation:**

We used our computer's built-in microphone and an earphone to record audio. Use of a better quality noise cancellation mic can result in improved performance.

- **Limited Dataset and Training Samples:**

The accuracy of the model is highly dependent on the quantity and quality of training data. A small or unbalanced dataset may lead to misclassification.

- **Replay Attack Vulnerability:**

Without advanced spoofing detection, the system may be vulnerable to replay attacks using pre-recorded voice samples.

Future Scope:

The future scopes of this project are-

- **Voice Spoofing Detection:**

To enhance security, future versions can include voice spoofing or replay attack detection mechanisms, ensuring that recorded or imitated voices cannot bypass the system.

- **Integration of Deep Learning Models:**

The system can be upgraded by incorporating advanced models like Convolutional Neural Networks (CNNs) or Recurrent Neural Networks (RNNs), which offer higher accuracy and better speaker discrimination compared to traditional SVM models.

- **Multi-language and Multi-factor Support:**

Expanding beyond Bengali, the system can support multiple languages, making it suitable for broader use. Additionally, integrating it with multi-factor authentication (e.g., combining voice with OTP or facial recognition) can provide stronger security.

- **Deployment on Mobile and Web Platforms:**

To enable remote and real-time banking access, the system can be deployed on mobile apps or web platforms, allowing users to authenticate securely from anywhere.

Conclusion:

In this project, we developed a voice-based authentication system that enhances security by using the natural uniqueness of the human voice. Traditional authentication methods like passwords or ID cards are often vulnerable to theft, loss, or duplication. Our system tackles this problem by requiring users to both type and speak their name and ID, and then verifying the spoken voice using machine learning techniques.

Using MFCC (Mel Frequency Cepstral Coefficients), we were able to extract meaningful features from the user's voice. These features were further processed with delta and delta-delta coefficients to capture speech dynamics, and then classified using RF (Random Forest). By comparing the spoken inputs to pre-trained models, the system can decide whether the speaker is truly the account holder.

One of the strengths of this system is that it is both speaker-dependent and speech-dependent, meaning it not only identifies who is speaking, but also confirms what is being said. This dual-layer check improves accuracy and adds an extra level of protection. The entire process, from recording to decision-making, is implemented using MATLAB, ensuring reliability in signal processing and classification tasks.

From our testing, it is clear that the system performs best in quiet environments, showing high accuracy and consistency in identifying users. While environmental noise remains a challenge, this project lays a strong foundation for future improvements such as noise reduction, real-time deployment, or multi-language support.

Overall, the project highlights how voice biometrics can serve as a secure, contactless, and user-friendly authentication method, suitable for high-security applications like banking or personal access control. It opens the door for smarter and safer identity verification solutions in a digital world where traditional methods are no longer enough.

References

1. Z. K. Abdul and A. K. Al-Talabani, "Mel Frequency Cepstral Coefficient and its Applications: A Review," in IEEE Access, vol. 10, pp. 122136-122158, 2022, doi: 10.1109/ACCESS.2022.3223444.
2. Ali, Ashraf & Abdullah, Hasanen & Fadhil, Mohammad. (2021). Voice recognition system using machine learning techniques. Materials Today: Proceedings. 10.1016/j.matpr.2021.04.075.
3. H. H. Nuha and A. Abo Absa, "Noise Reduction and Speech Enhancement Using Wiener Filter," 2022 International Conference on Data Science and Its Applications (ICoDSA), Bandung, Indonesia, 2022, pp. 177-180, doi: 10.1109/ICoDSA55874.2022.9862912.
4. Jalil, Madiha & Butt, Faran & Malik, Ahmed. (2013). Short-time energy, magnitude, zero crossing rate and autocorrelation measurement for discriminating voiced and unvoiced segments of speech signals. Technol. Adv. Electric. Electron. Comp. Eng. (TAEECE). 208-212. 10.1109/TAEECE.2013.6557272.
5. Audrey Cheong
(2025). confusionmatStats(group,grouphat) (<https://www.mathworks.com/matlabcentral/fileexchange/46035-confusionmatstats-group-grouphat>), MATLAB Central File Exchange. Retrieved July 31, 2025.